



Рафеенко Е.Д.

Web- программирование

Реактивное программирование. Библиотека RxJS

Содержание

- ▶ Введение
- ▶ Создание и сборка JS - проекта
- ▶ Observables & RxJS
- ▶ Операторы RxJS



Введение

Реактивное программирование — это парадигма асинхронного программирования, связанная с потоками данных (streams) и распространением изменений.

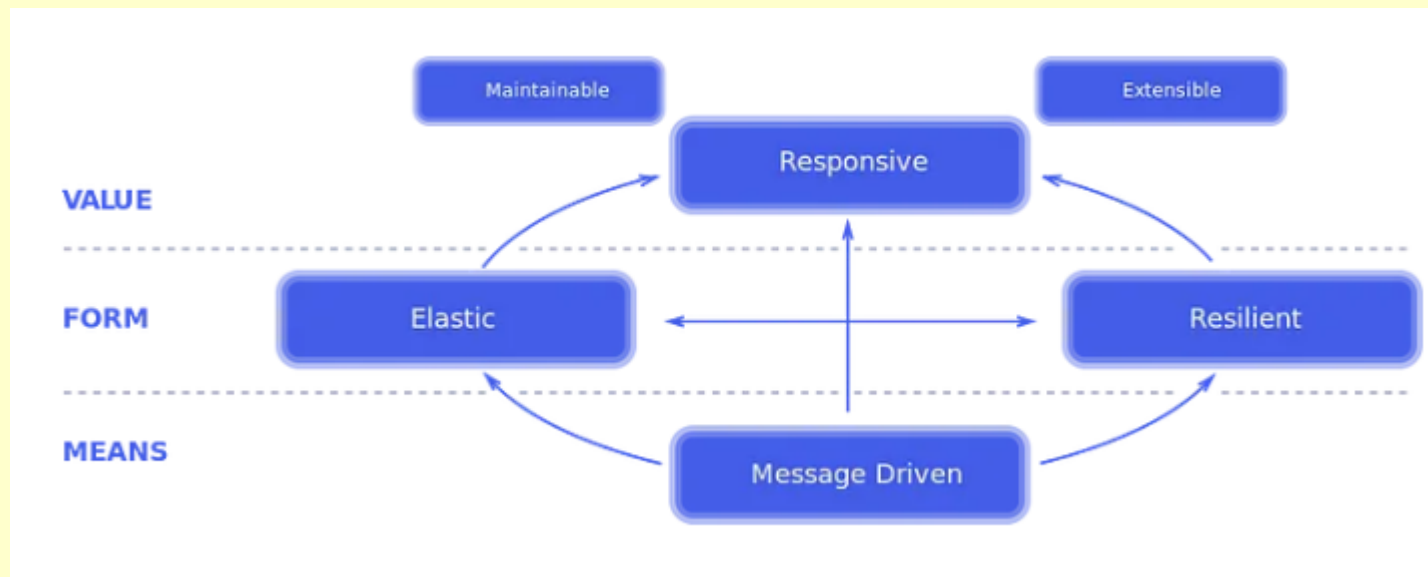
Эта парадигма полезна при обработке больших объемов асинхронных данных.

Web-приложениям нужны быстрые отклики и возможность обрабатывать данные из разных источников одновременно.

В 2014 году был опубликован [Reactive Manifesto](#), в котором изложены необходимость и принципы реактивного программирования.



Введение



Приложение реагирует своевременно, если это вообще возможно;
Приложение реагирует на сбои;
Приложение остается отзывчивым при различной рабочей нагрузке;
Реактивные системы полагаются на асинхронную передачу сообщений.



RxJS

RxJS (Reactive Extensions for JavaScript) — это библиотека для реактивного программирования с использованием наблюдаемых объектов (observables), которая упрощает создание асинхронного кода или callback-based кода.

<https://rxjs.dev/guide/overview>

<https://www.learnrxjs.io/>



Создание JS - проекта

1. Создать папку для нового проекта - например, в командной строке выполнить:

```
mkdir DemoRxJS
```

2. Перейти в папку проекта:

```
cd DemoRxJs
```

3. Инициализировать проект:

```
npm init -y
```

В корневой папке будет создан файл package.json



Конфигурация проекта

package.json – файл конфигурации проекта

```
{  
  "dependencies": {  
    Пакеты, необходимые для работы приложения (включая Angular)  
  
  },  
  "devDependencies": {  
    Пакеты, используемые в процессе разработки (например, компиляторы и  
    тестовые фреймворки)  
  
  },  
  "scripts": {  
    Команды, используемые для компиляции, тестирования и запуска  
    приложения  
  
  }  
}
```



Конфигурация проекта

package.json – пример

```
{  
  "name": "demorxjs1",  
  "scripts": {  
    "dev": "webpack --mode development",  
    "start": "node server.js" },  
  "dependencies": {  
    "express": "^4.18.2",  
    "rxjs": "~7.8.0"  
  },  
  "devDependencies": {  
    "html-webpack-plugin": "^5.5.0",  
    "ts-loader": "^9.5.1",  
    "typescript": "^5.4.5",  
    "webpack": "^5.91.0",  
    "webpack-cli": "^5.0.1",  
  }  
}
```



Выполнение проекта

Загрузить пакеты, указанные в `package.json`, и установить их в папку `node_modules`:

```
npm install
```

Запуск команд из раздела `scripts`:

```
npm run dev
```

```
npm run start
```

Файл `package-lock.json` ни при каких условиях нельзя удалять или редактировать, он обновляется автоматически при установке или удалении пакетов.



Webpack

Что такое Webpack?

Webpack — это инструмент, позволяющий скомпилировать JavaScript модули в единый JS-файл. Webpack также известен как **сборщик модулей**.

Webpack создает граф зависимостей, затем собирает модули в правильном порядке в один или более бандл (bundle), на который может ссылаться файл index.html.

Webpack модули оптимизирует под режим development или production



Webpack

Основной задачей webpack - является анализ модулей, их преобразование и объединение в один или более файл (бандл).
Конфигурационный файл Webpack - `webpack.config.js` хранится в корневой директории проекта.

В нем нужно задать:

Точку входа в приложение.

Преобразования, которые необходимо выполнить.

Место, в которое следует поместить сформированный файл.



Webpack

Загрузчики (loaders) - расширяют Webpack, позволяя обрабатывать не-JS файлы: CSS, изображения, TS и др.

- принимают содержимое файлов, преобразуют их необходимым образом и включают результат преобразования в общую сборку.

Для импорта стилей используется **css-loader**.

Чтобы включить CSS файлы в DOM нужен **style-loader**.

Загрузчик **ts-loader** позволяет работать с ts-файлами.

ts-loader использует tsc , компилятор TypeScript , и опирается на конфигурацию tsconfig.json .

Преобразование JavaScript предыдущего поколения в современный JavaScript с осуществляется с помощью Babel.
Для этого используется **babel-loader**.



Webpack

Плагины - плагины позволяют выполнять задачи после сборки бандла.

Плагин HTMLWebpackPlugin автоматически создает HTML-файл с уже подключенным собранным скриптом.

HtmlWebpackPlugin создаст новый файл index.html в выходной директории (dist) и добавит в него ссылку на скрипт:

```
<script src='main.js'></script>
```

Сервер для разработки - webpack-dev-server.

Вместо создания директории dist, он хранит данные в памяти и обрабатывает их на локальном сервере, поддерживает живую перезагрузку. Это означает, что при любом изменении webpack-dev-server пересоберет файлы и перезапустит браузер.



RxJS - Stream

Stream — это последовательность текущих событий, упорядоченных во времени.

Stream производит (emits) 3 типа объекта:

типизированное значение (data value),

Ошибку (error),

Сигнал завершения (termination mark).

Эти генерируемые события фиксируются только асинхронно.

Для этого определяются функции, которые вызываются и им передается одно из трех соответствующих выходных событий в качестве параметра.



RxJS – основные понятия

Observable: поток значений или событий.

Observer: это набор функций, которые знают, как прослушивать значения, предоставленные Observable.

Subscription: представляет выполнение Observable (полезно для отмены выполнения)

Operators: это функции, которые обеспечивают функциональный стиль программирования для работы с потоками (map, filter, concat, reduce и т. д).



Pull & Push в JavaScript

Observables – это *lazy push* коллекции.

Протоколы *push* и *pull* описывают как производитель данных (*Producer*) взаимодействует с потребителем данных (*Consumer*).

В *pull* системах *Consumer* определяет, когда он получает данные от *Producer*. Сам *Producer* не знает, когда данные будут доставлены *Consumer*.

Пример. Любая JavaScript функция — *pull* система.

Функция `generator` и `iterator (function*)` другой тип *pull* системы в JavaScript.



Pull & Push в JavaScript

В системах *push* *Producer* определяет, когда отправлять данные *Consumer*. *Consumer* не знает, когда он получит эти данные.

Promise — самый распространенный тип системы *push* в JavaScript.

Promise (*Producer*) предоставляет значение зарегистрированным *callbacks* (*Consumer*);

Promise отвечает за точное определение того, когда это значение «проталкивается» в *callbacks*.

Observables - новая *push* система в JavaScript.

Observable — это производитель *нескольких значений*, «проталкивающий» их к наблюдателям (потребителям).



Observable

В среде реактивного программирования наблюдаемый объект (Observable) отвечает за отправку данных подписчикам (*push*) .
Observable можно создавать с помощью `new Observable`.

```
const observable =  
new Observable(function subscribe(subscriber)  
{  
  
  }) ;
```

На наблюдаемый объект нужно подписаться:

```
observable.subscribe(observer) ;
```



Observables - создание

Observables могут предоставлять значения синхронно или асинхронно.

Observables могут «возвращать» несколько значений с течением времени.



Observers

Observer — это потребитель значений, предоставляемых Observable.

Observer — это набор функций, по одной для каждого типа уведомления, доставляемого Observable:

next, error, complete.

```
const observer = {  
  next: x => console.log('Observer got a next value: ' + x),  
  error: err => console.error('Observer got an error: ' + err),  
  complete: () => console.log('Observer got a complete notification'),  
};
```

```
observable.subscribe(observer);
```



Observables

На любую подписку создается отдельный поток значений для большинства Observable.

Есть Observable, распределяющий события по всем подписчикам (оператор `fromEvent`).

Observable могут быть *холодными* и *горячими*.

Холодные создают независимые потоки под каждую подписку.

Горячие разделяют поток друг с другом (Subjects).



Observables

Холодный поток (**Cold Observable**)

создаёт данные для каждого подписчика заново;
каждый subscriber получает свою собственную последовательность.

Горячий Observable (**Hot Observable**)

уже “идёт”, и подписчики получают только текущие значения;

не создаёт поток заново для каждого подписчика.

Пример — Subject.



Subjects

Subject — это особый тип Observable, который позволяет передавать значения многим наблюдателям (multicasting).

Каждый Subject – Observable.

Можно подписаться на него, предоставив Observer, который будет получать значения.

Каждый Subject – Observer.

У него есть методы `next(v)`, `error(e)`, `complete()`.

Чтобы передать новое значение в Subject, нужно вызвать `next(value)`, и оно будет передано Observers, зарегистрированным для прослушивания Subject.

Т.к Subject является Observer, это также означает, что можно предоставить Subject в качестве аргумента в метод `subscribe()` любого Observable объекта.



Операторы RxJS

Операторы — это функции, обеспечивающие декларативное решение сложных асинхронных задач.

Существует 2 типа операторов.

Pipeable operators (конвейерные операторы).

Pipeable operator — это функция, которая принимает Observable в качестве входных данных и возвращает другой Observable. Исходный Observable остается неизменным (pure operation).

Creation operators (Операторы создания) — это функции, которые можно вызывать для создания нового Observable.



Операторы создания

Чаще всего observables создаются с помощью функций:

of() — возвращает Observable на основе своих аргументов, а затем завершает работу.

from() — возвращает Observable на основе, например, массива.

fromEvent() — превращает событие в Observable.

interval() — возвращает Observable, который выдает значение с заданным интервалом.

...



Операторы создания

Код без observable:

```
const array = [1, 2, 'three', true];

array.forEach(el => {
  console.log(el)
});
```

Код с observable:

```
import { from } from 'rxjs';

const array = [1, 2, 'three', true];

from(array)
  .subscribe((val) => console.log(val));
```

array → Observable
forEach → subscribe
callback → Observer



Оператор pipe

Конвейерные операторы — это те, которые могут быть переданы в Observables с использованием синтаксиса:

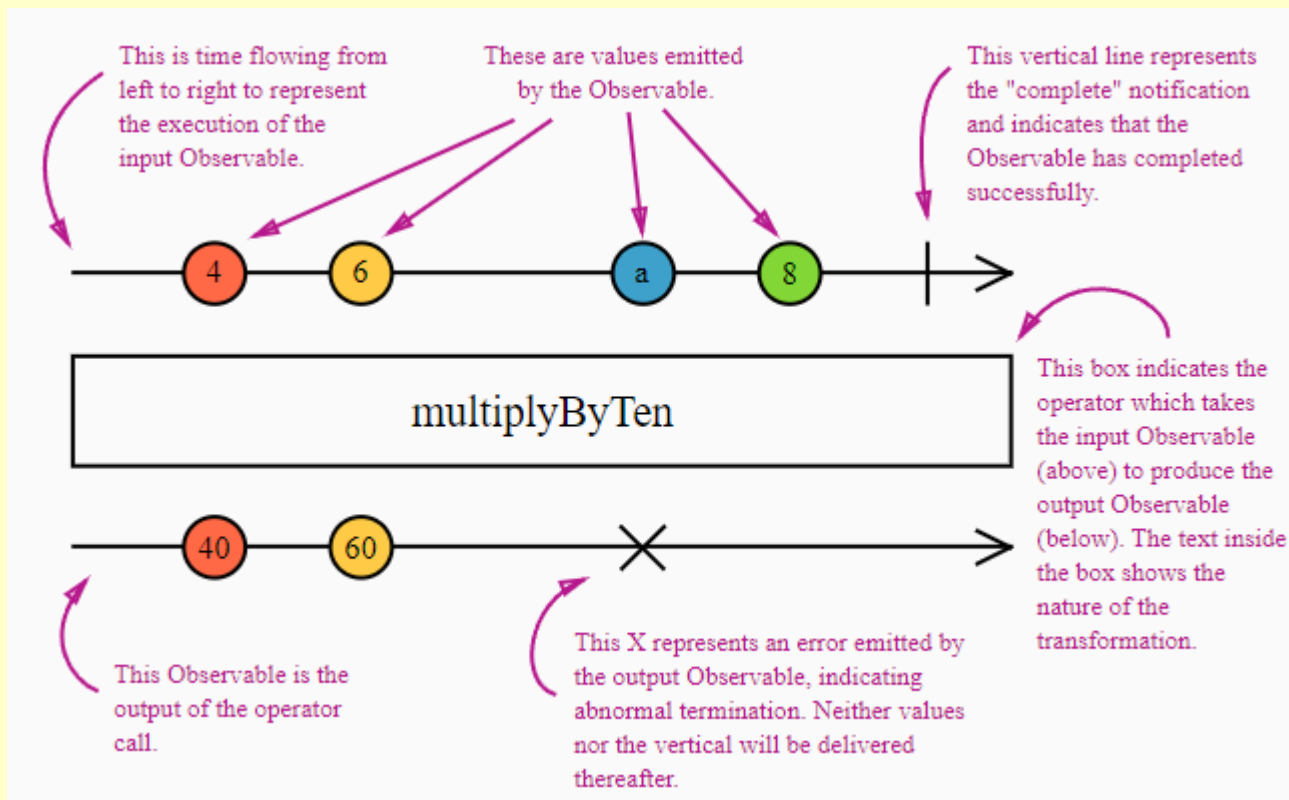
```
observableInstance.pipe(operator())
```

При вызове они не изменяют существующий экземпляр Observable. Вместо этого они возвращают новый Observable, логика подписки которого основана на первом Observable. Например, операторы `map()`, `filter(...)`.



Marble Diagrams

Мраморные диаграммы — это визуальное представление того, как работают операторы, и они включают входные наблюдаемые объекты, оператор и его параметры, а также выходные наблюдаемые объекты.





Виды Subjects

BehaviorSubject, ReplaySubject и AsyncSubject

BehaviorSubject - хранит последнее значение и сразу отдаёт его новому подписчику.

Можно использовать для хранения состояния.

ReplaySubject хранит несколько последних значений (или все) и отдаёт их новому подписчику.

Можно использовать для логгирования событий, сохранения истории сообщений в чате, восстановления состояния при поздней подписке

AsyncSubject отдаёт только последнее значение, и только после `complete()`.

Можно использовать для асинхронных операций, которые важны только по завершении, загрузки конфигурации, одноразовых вычислениях.



Multicast

Мультикаст (multicast) в контексте RxJS — это способ, при котором один источник данных рассылает одни и те же значения сразу нескольким подписчикам.

В данном случае поток работает один раз, а подписчиков много. Все подписчики получают одни и те же значения, синхронно.

Оператор **share**.

share() превращает холодный Observable в горячий мультикаст-Observable:

- поток создаётся один раз, а не заново для каждого подписчика;
- все подписчики получают одни и те же значения;
- поток начинается при первой подписке;
- поток останавливается, когда нет подписчиков



Multicast

Например,

```
ajax.getJSON('/items').pipe(  
  share()  
)  
data$.subscribe(v => console.log('A:', v));  
data$.subscribe(v => console.log('B:', v));
```

Сервер получит **1 запрос**, а оба подписчика получают один и тот же результат.



High order observables

Observables выдают:

простые значения, такие как строки и числа,
или Observables – это high-order Observables.

Например,

```
const fileObservable = urlObservable.pipe(map((url) =>  
http.get(url)));
```

`http.get(url)` выдает Observable для каждого url.

Обычно high-order Observables преобразуются в обычные Observables. Это операторы типа `join`: `concatAll()`, `mergeAll()`, `switchAll()`.



Операторы типа join

concatAll() – преобразует high-order Observable в Observable первого порядка, объединяя внутренние Observable по порядку.

mergeAll() – преобразует high-order Observable в Observable первого порядка, который одновременно доставляет все значения, испускаемые внутренними Observable.

switchAll() – преобразует high-order Observable в Observable первого порядка, производя значения только из самой последней наблюдаемой последовательности.



Операторы трансформации

concatMap() (*Делай по очереди*) – трансформирует каждое значение в Observable, затем объединяет все эти внутренние Observable с помощью *concatAll*.

mergeMap() (*Делай всё параллельно*) – трансформирует каждое значение в Observable, затем объединяет все эти внутренние Observable с помощью *mergeAll*.

switchMap() (*Отменяй старое, делай новое*) – трансформирует каждое значение в Observable, затем объединяет все эти внутренние Observable с помощью *switchAll*.

Т.е.

`concatMap = map + concatAll`

`mergeMap = map + mergeAll`

`switchMap = map + switchAll`



Операторы трансформации

Например,

`switchAll` — «переключиться на последний поток»: отменяет предыдущий внутренний `Observable`; подписывается только на последний. Можно использовать для поиска, автокомплита, запросов.

```
source$  
  .pipe(  
    map(v => ajax.getJSON(`/search?q=${v}`)),  
    switchAll()  
  )  
  .subscribe(console.log);
```

или кратко с помощью `switchMap`:

```
source$  
  .pipe(  
    switchMap(v => ajax.getJSON(`/search?q=${v}`))  
  )  
  .subscribe(console.log);
```



Операторы трансформации

exhaustMap — «игнорируй новые потоки, пока выполняется текущий»

если внутренний Observable уже выполняется, то новые значения игнорируются пока текущий не завершится.

```
loginClicks$.pipe(  
  exhaustMap(() => ajax.post('/login'))  
)
```

Можно использовать в форме логина, для обработки кнопки «Отправить», для защиты от двойных кликов, предотвращения повторных запросов.



Операторы типа join

`combineLatest` — это оператор RxJS, который ждёт, пока каждый Observable выдаст хотя бы одно значение:

```
combineLatest([items$, search$, sort$]);
```

затем каждый раз, когда любой этих Observable выдаёт новое значение, он собирает последние значения всех потоков и передаёт их дальше как массив: `[items, search, sort]`